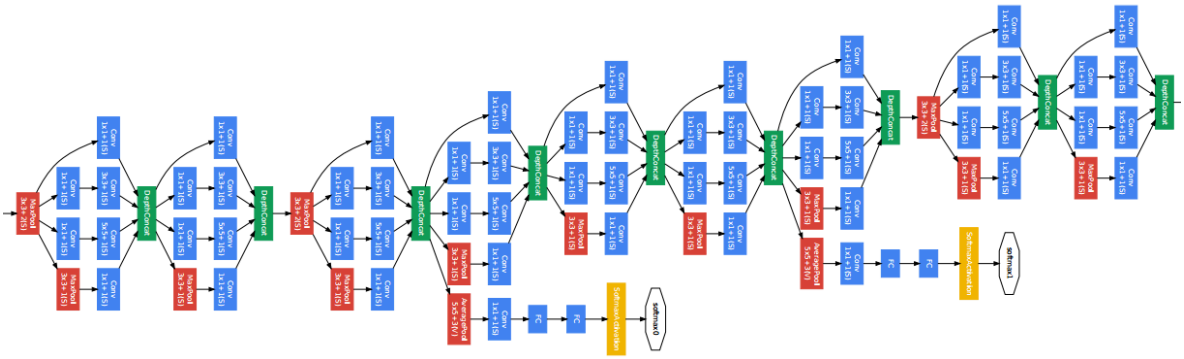


In [ ]:
```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
```

In [ ]:
```
!kaggle datasets download samuelcortinhas/sports-balls-multiclass-image-classi
```

```
Warning: Your Kaggle API key is readable by other users on this system! To fix
this, you can run 'chmod 600 /root/.kaggle/kaggle.json'
Dataset URL: https://www.kaggle.com/datasets/samuelcortinhas/sports-balls-multi
class-image-classification
License(s): CC0-1.0
Downloading sports-balls-multiclass-image-classification.zip to /content
 98% 392M/401M [00:00<00:00, 435MB/s]
100% 401M/401M [00:00<00:00, 469MB/s]
```

In [ ]:
```
import tensorflow as tf
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import seaborn as sns
import os
```

In [ ]:
```
# Set up directory and parameters
train_dir = '/content/train'
img_size = (224, 224)
batch_size = 32
num_classes = 15

seed = 123
random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
```

In [ ]:
```
train_dir = '/content/train'

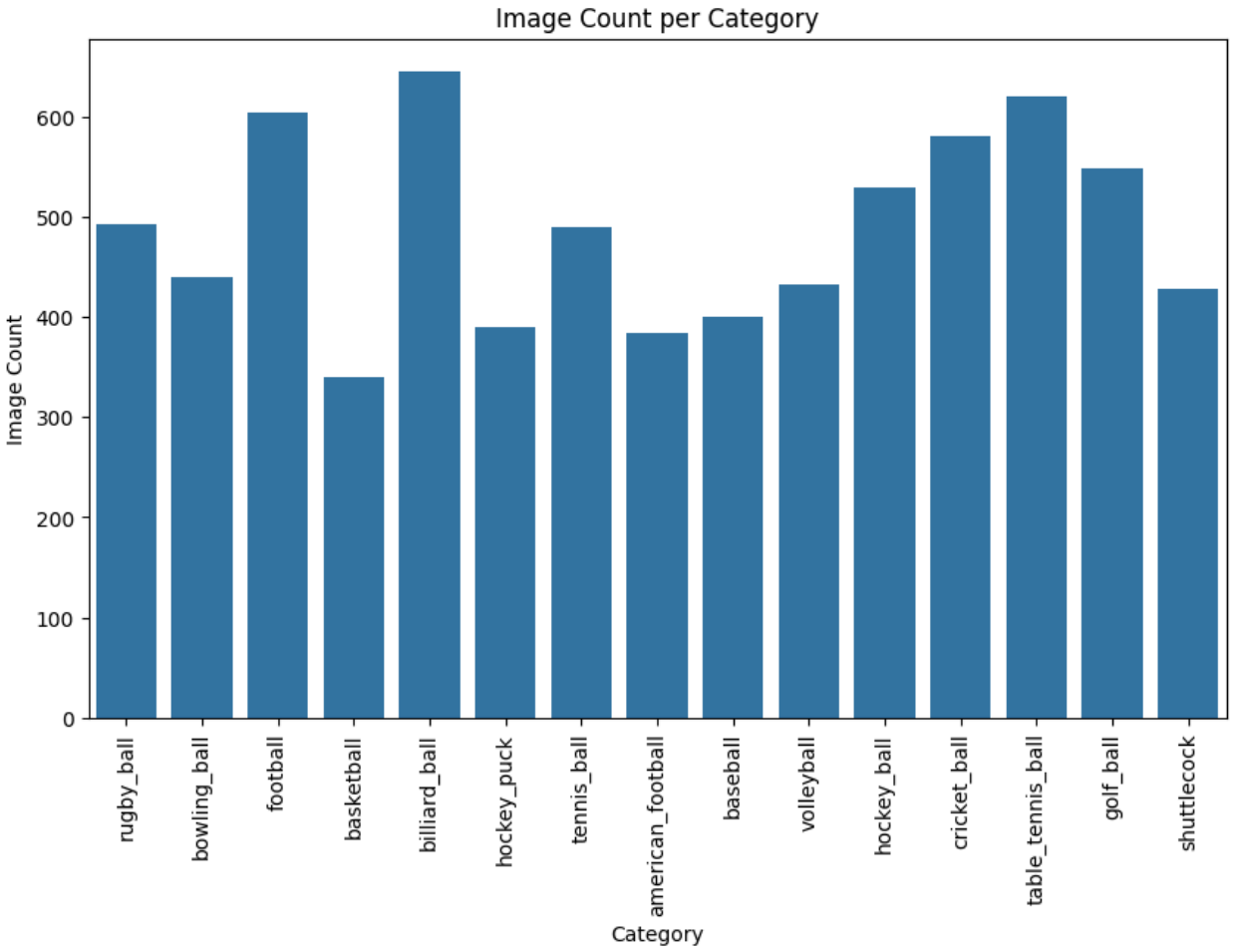
data = {'Category': [], 'Image Count': []}
```

In [ ]:
```
for sub_dir in os.listdir(train_dir):
  sub_dir_path = os.path.join(train_dir, sub_dir)
  data['Category'].append(sub_dir)
```

```
  data['Image Count'].append(len(os.listdir(sub_dir_path)))

df = pd.DataFrame(data)

plt.figure(figsize=(10, 6))
sns.barplot(x='Category', y='Image Count', data=df)
plt.xticks(rotation=90)
plt.xlabel('Category')
plt.ylabel('Image Count')
plt.title('Image Count per Category')
plt.show()
```



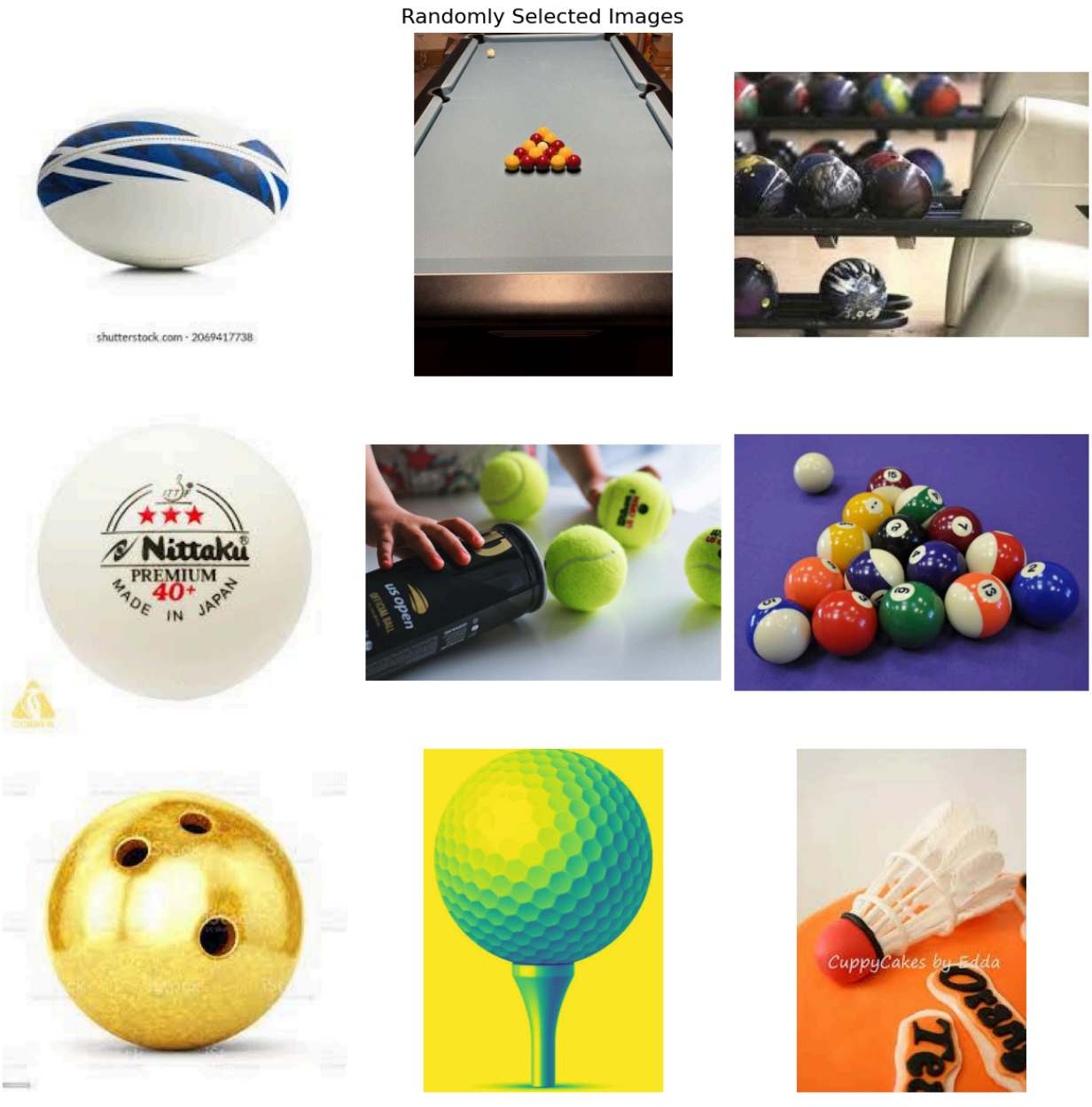
In [ ]:
```
# Directory path
train_dir = '/content/train'

# Collect all image file paths
all_images = []
for sub_dir in os.listdir(train_dir):
    sub_dir_path = os.path.join(train_dir, sub_dir)
    if os.path.isdir(sub_dir_path):
        for filename in os.listdir(sub_dir_path):
            file_path = os.path.join(sub_dir_path, filename)
            if os.path.isfile(file_path):
                all_images.append(file_path)
```

```
# Randomly select 9 images
random_images = random.sample(all_images, 9)

# Plotting the images in a 3x3 grid
plt.figure(figsize=(12, 12))
for i, image_path in enumerate(random_images):
    img = mpimg.imread(image_path)
    plt.subplot(3, 3, i + 1)
    plt.imshow(img)
    plt.axis('off')

plt.suptitle('Randomly Selected Images', fontsize=16)
plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]:
```
# Preprocess the datasets so each label is triplicated for multi-output model
def process_multi_output(image, label):
    return image, (label, label, label)

train_dataset = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset='training',
    seed=123,
    image_size=img_size,
    batch_size=batch_size,
)
train_dataset = train_dataset.map(process_multi_output)

validation_dataset = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset='validation',
    seed=123,
    image_size=img_size,
    batch_size=batch_size,
)
validation_dataset = validation_dataset.map(process_multi_output)

# Prefetch for performance
AUTOTUNE = tf.data.AUTOTUNE
train_dataset = train_dataset.prefetch(buffer_size=AUTOTUNE)
validation_dataset = validation_dataset.prefetch(buffer_size=AUTOTUNE)
```

```
Found 7328 files belonging to 15 classes.
Using 5863 files for training.
Found 7328 files belonging to 15 classes.
Using 1465 files for validation.
```

In [ ]:

No description has been provided for this image

In [ ]:
```
import tensorflow as tf
from tensorflow.keras import layers, models
```

In [ ]:
```
# Inception model as defined in the paper
def inception_module(x, filters_1x1, filters_3x3_reduce, filters_3x3, filters_
    # 1x1 convolution
    conv_1x1 = layers.Conv2D(filters_1x1, (1, 1), padding='same', activation='

    # 3x3 convolution
    conv_3x3 = layers.Conv2D(filters_3x3_reduce, (1, 1), padding='same', activ
    conv_3x3 = layers.Conv2D(filters_3x3, (3, 3), padding='same', activation='

    # 5x5 convolution
```

```
    conv_5x5 = layers.Conv2D(filters_5x5_reduce, (1, 1), padding='same', activ
    conv_5x5 = layers.Conv2D(filters_5x5, (5, 5), padding='same', activation='

    # Max pooling followed by 1x1 convolution
    pool_proj = layers.MaxPooling2D((3, 3), strides=(1, 1), padding='same')(x)
    pool_proj = layers.Conv2D(filters_pool_proj, (1, 1), padding='same', activ

    # Concatenate filters from all branches
    output = layers.concatenate([conv_1x1, conv_3x3, conv_5x5, pool_proj], axi

    return output
```

In [ ]:
```
# Auxiliary classifier for training (deep supervision)
def auxiliary_classifier(x, num_classes):
  aux = layers.AveragePooling2D((5, 5), strides=(3, 3))(x)
  aux = layers.Conv2D(128, (1, 1), padding='same', activation='relu')(aux)
  aux = layers.Flatten()(aux)
  aux = layers.Dense(1024, activation='relu')(aux)
  aux = layers.Dropout(0.7)(aux)
  aux = layers.Dense(num_classes, activation='softmax')(aux)
  return aux
```

In [ ]:
```
# Build the GoogLeNet model with two auxiliary classifiers
def build_googlenet(input_shape, num_classes):
  input_layer = layers.Input(shape = input_shape)

  x = layers.Rescaling(1./255)(input_layer)

  # Initial Convolution and MaxPooling
  x = layers.Conv2D(64, (7, 7), strides=(2, 2), padding='same', activation='re
  x = layers.MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x)

  # Local Response Normalization (can be replaced with BatchNormalization)
  x = layers.BatchNormalization(name='norm1')(x)

  # Second Convolution and MaxPooling
  x = layers.Conv2D(192, (3, 3), padding='same', activation='relu')(x)
  x = layers.MaxPooling2D((3,3), strides=(2,2), padding='same')(x)

  # Inception modules 1 to 4
  x = inception_module(x, 64, 96, 128, 16, 32, 32) # Inception 3a
  x = inception_module(x, 128, 128, 192, 32, 96, 64) # Inception 3b

  x = layers.MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x) # MaxPool

  x = inception_module(x, 192, 96, 208, 16, 48, 64) # Inception 4a

  # First auxiliary classifier after the 4th inception model
  aux1 = auxiliary_classifier(x, num_classes)

  x = inception_module(x, 160, 112, 224, 24, 64, 64) # Inception 4b
  x = inception_module(x, 128, 128, 256, 24, 64, 64) # Inception 4c
  x = inception_module(x, 112, 144, 288, 32, 64, 64) # Inception 4d
```

```
    # Second auxiliary classifier after the 7th inception model
    aux2 = auxiliary_classifier(x, num_classes)

    x = inception_module(x, 256, 160, 320, 32, 128, 128) # Inception 4e

    x = layers.MaxPooling2D((3, 3), strides=(2, 2), padding='same')(x) # MaxPool

    # Inception modules 8 and 9
    x = inception_module(x, 256, 160, 320, 32, 128, 128) # Inception 5a
    x = inception_module(x, 384, 192, 384, 48, 128, 128) # Inception 5b

    # Global Average Pooling and Dropout
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.4)(x)

    # Final output layer (main classifier)
    output_layer = layers.Dense(num_classes, activation='softmax')(x)

    # Define the model with inputs and three output (main + 2 auxiliary)
    model = models.Model(inputs = input_layer, outputs = [output_layer, aux1, au

    return model
```

In [ ]:
```
# Model parameters
input_shape = (224, 224, 3) # ImageNet image size
num_classes = 15 # Number of classes for classification

# Build the model
googlenet = build_googlenet(input_shape, num_classes)
```

In [ ]:
```
# Print the model summary
googlenet.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
rescaling (Rescaling)	(None, 224, 224, 3)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 112, 112, 64)	9,472	rescaling[0][0]
max_pooling2d (MaxPooling2D)	(None, 56, 56, 64)	0	conv2d[0][0]
norm1 (BatchNormalizatio…	(None, 56, 56, 64)	256	max_pooling2d[0]…
conv2d_1 (Conv2D)	(None, 56, 56, 192)	110,784	norm1[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 28, 28, 192)	0	conv2d_1[0][0]
conv2d_3 (Conv2D)	(None, 28, 28, 96)	18,528	max_pooling2d_1[…
conv2d_5 (Conv2D)	(None, 28, 28, 16)	3,088	max_pooling2d_1[…
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 192)	0	max_pooling2d_1[…
conv2d_2 (Conv2D)	(None, 28, 28, 64)	12,352	max_pooling2d_1[…
conv2d_4 (Conv2D)	(None, 28, 28, 128)	110,720	conv2d_3[0][0]
conv2d_6 (Conv2D)	(None, 28, 28, 32)	12,832	conv2d_5[0][0]
conv2d_7 (Conv2D)	(None, 28, 28, 32)	6,176	max_pooling2d_2[…
concatenate (Concatenate)	(None, 28, 28, 256)	0	conv2d_2[0][0], conv2d_4[0][0], conv2d_6[0][0], conv2d_7[0][0]
conv2d_9 (Conv2D)	(None, 28, 28, 128)	32,896	concatenate[0][0]
conv2d_11 (Conv2D)	(None, 28, 28, 32)	8,224	concatenate[0][0]

max_pooling2d_3 (MaxPooling2D)	(None, 28, 28, 256)	0	concatenate[0][0]
conv2d_8 (Conv2D)	(None, 28, 28, 128)	32,896	concatenate[0][0]
conv2d_10 (Conv2D)	(None, 28, 28, 192)	221,376	conv2d_9[0][0]
conv2d_12 (Conv2D)	(None, 28, 28, 96)	76,896	conv2d_11[0][0]
conv2d_13 (Conv2D)	(None, 28, 28, 64)	16,448	max_pooling2d_3[…
concatenate_1 (Concatenate)	(None, 28, 28, 480)	0	conv2d_8[0][0], conv2d_10[0][0], conv2d_12[0][0], conv2d_13[0][0]
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 480)	0	concatenate_1[0]…
conv2d_15 (Conv2D)	(None, 14, 14, 96)	46,176	max_pooling2d_4[…
conv2d_17 (Conv2D)	(None, 14, 14, 16)	7,696	max_pooling2d_4[…
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 480)	0	max_pooling2d_4[…
conv2d_14 (Conv2D)	(None, 14, 14, 192)	92,352	max_pooling2d_4[…
conv2d_16 (Conv2D)	(None, 14, 14, 208)	179,920	conv2d_15[0][0]
conv2d_18 (Conv2D)	(None, 14, 14, 48)	19,248	conv2d_17[0][0]
conv2d_19 (Conv2D)	(None, 14, 14, 64)	30,784	max_pooling2d_5[…
concatenate_2 (Concatenate)	(None, 14, 14, 512)	0	conv2d_14[0][0], conv2d_16[0][0], conv2d_18[0][0], conv2d_19[0][0]
conv2d_22 (Conv2D)	(None, 14, 14, 112)	57,456	concatenate_2[0]…
conv2d_24 (Conv2D)	(None, 14, 14, 24)	12,312	concatenate_2[0]…
max_pooling2d_6	(None, 14, 14,	0	concatenate_2[0]…

(MaxPooling2D)	512)		
conv2d_21 (Conv2D)	(None, 14, 14, 160)	82,080	concatenate_2[0]…
conv2d_23 (Conv2D)	(None, 14, 14, 224)	226,016	conv2d_22[0][0]
conv2d_25 (Conv2D)	(None, 14, 14, 64)	38,464	conv2d_24[0][0]
conv2d_26 (Conv2D)	(None, 14, 14, 64)	32,832	max_pooling2d_6[…
concatenate_3 (Concatenate)	(None, 14, 14, 512)	0	conv2d_21[0][0], conv2d_23[0][0], conv2d_25[0][0], conv2d_26[0][0]
conv2d_28 (Conv2D)	(None, 14, 14, 128)	65,664	concatenate_3[0]…
conv2d_30 (Conv2D)	(None, 14, 14, 24)	12,312	concatenate_3[0]…
max_pooling2d_7 (MaxPooling2D)	(None, 14, 14, 512)	0	concatenate_3[0]…
conv2d_27 (Conv2D)	(None, 14, 14, 128)	65,664	concatenate_3[0]…
conv2d_29 (Conv2D)	(None, 14, 14, 256)	295,168	conv2d_28[0][0]
conv2d_31 (Conv2D)	(None, 14, 14, 64)	38,464	conv2d_30[0][0]
conv2d_32 (Conv2D)	(None, 14, 14, 64)	32,832	max_pooling2d_7[…
concatenate_4 (Concatenate)	(None, 14, 14, 512)	0	conv2d_27[0][0], conv2d_29[0][0], conv2d_31[0][0], conv2d_32[0][0]
conv2d_34 (Conv2D)	(None, 14, 14, 144)	73,872	concatenate_4[0]…
conv2d_36 (Conv2D)	(None, 14, 14, 32)	16,416	concatenate_4[0]…
max_pooling2d_8 (MaxPooling2D)	(None, 14, 14, 512)	0	concatenate_4[0]…
conv2d_33 (Conv2D)	(None, 14, 14, 112)	57,456	concatenate_4[0]…

conv2d_35 (Conv2D)	(None, 14, 14, 288)	373,536	conv2d_34[0][0]
conv2d_37 (Conv2D)	(None, 14, 14, 64)	51,264	conv2d_36[0][0]
conv2d_38 (Conv2D)	(None, 14, 14, 64)	32,832	max_pooling2d_8[…
concatenate_5 (Concatenate)	(None, 14, 14, 528)	0	conv2d_33[0][0], conv2d_35[0][0], conv2d_37[0][0], conv2d_38[0][0]
conv2d_41 (Conv2D)	(None, 14, 14, 160)	84,640	concatenate_5[0]…
conv2d_43 (Conv2D)	(None, 14, 14, 32)	16,928	concatenate_5[0]…
max_pooling2d_9 (MaxPooling2D)	(None, 14, 14, 528)	0	concatenate_5[0]…
conv2d_40 (Conv2D)	(None, 14, 14, 256)	135,424	concatenate_5[0]…
conv2d_42 (Conv2D)	(None, 14, 14, 320)	461,120	conv2d_41[0][0]
conv2d_44 (Conv2D)	(None, 14, 14, 128)	102,528	conv2d_43[0][0]
conv2d_45 (Conv2D)	(None, 14, 14, 128)	67,712	max_pooling2d_9[…
concatenate_6 (Concatenate)	(None, 14, 14, 832)	0	conv2d_40[0][0], conv2d_42[0][0], conv2d_44[0][0], conv2d_45[0][0]
max_pooling2d_10 (MaxPooling2D)	(None, 7, 7, 832)	0	concatenate_6[0]…
conv2d_47 (Conv2D)	(None, 7, 7, 160)	133,280	max_pooling2d_10…
conv2d_49 (Conv2D)	(None, 7, 7, 32)	26,656	max_pooling2d_10…
max_pooling2d_11 (MaxPooling2D)	(None, 7, 7, 832)	0	max_pooling2d_10…
conv2d_46 (Conv2D)	(None, 7, 7, 256)	213,248	max_pooling2d_10…
conv2d_48 (Conv2D)	(None, 7, 7, 320)	461,120	conv2d_47[0][0]
conv2d_50 (Conv2D)	(None, 7, 7, 128)	102,528	conv2d_49[0][0]

conv2d_51 (Conv2D)	(None, 7, 7, 128)	106,624	max_pooling2d_11…
concatenate_7 (Concatenate)	(None, 7, 7, 832)	0	conv2d_46[0][0], conv2d_48[0][0], conv2d_50[0][0], conv2d_51[0][0]
conv2d_53 (Conv2D)	(None, 7, 7, 192)	159,936	concatenate_7[0]…
conv2d_55 (Conv2D)	(None, 7, 7, 48)	39,984	concatenate_7[0]…
max_pooling2d_12 (MaxPooling2D)	(None, 7, 7, 832)	0	concatenate_7[0]…
average_pooling2d (AveragePooling2D)	(None, 4, 4, 512)	0	concatenate_2[0]…
average_pooling2d_1 (AveragePooling2D)	(None, 4, 4, 528)	0	concatenate_5[0]…
conv2d_52 (Conv2D)	(None, 7, 7, 384)	319,872	concatenate_7[0]…
conv2d_54 (Conv2D)	(None, 7, 7, 384)	663,936	conv2d_53[0][0]
conv2d_56 (Conv2D)	(None, 7, 7, 128)	153,728	conv2d_55[0][0]
conv2d_57 (Conv2D)	(None, 7, 7, 128)	106,624	max_pooling2d_12…
conv2d_20 (Conv2D)	(None, 4, 4, 128)	65,664	average_pooling2…
conv2d_39 (Conv2D)	(None, 4, 4, 128)	67,712	average_pooling2…
concatenate_8 (Concatenate)	(None, 7, 7, 1024)	0	conv2d_52[0][0], conv2d_54[0][0], conv2d_56[0][0], conv2d_57[0][0]
flatten (Flatten)	(None, 2048)	0	conv2d_20[0][0]
flatten_1 (Flatten)	(None, 2048)	0	conv2d_39[0][0]
global_average_poo… (GlobalAveragePool…	(None, 1024)	0	concatenate_8[0]…
dense (Dense)	(None, 1024)	2,098,176	flatten[0][0]
dense_2 (Dense)	(None, 1024)	2,098,176	flatten_1[0][0]
dropout_2 (Dropout)	(None, 1024)	0	global_average_p…
dropout (Dropout)	(None, 1024)	0	dense[0][0]
dropout_1 (Dropout)	(None, 1024)	0	dense_2[0][0]
dense_4 (Dense)	(None, 15)	15,375	dropout_2[0][0]

dense_1 (Dense)	(None, 15)	15,375	dropout[0][0]
dense_3 (Dense)	(None, 15)	15,375	dropout_1[0][0]

Total params: 10,345,501 (39.46 MB)

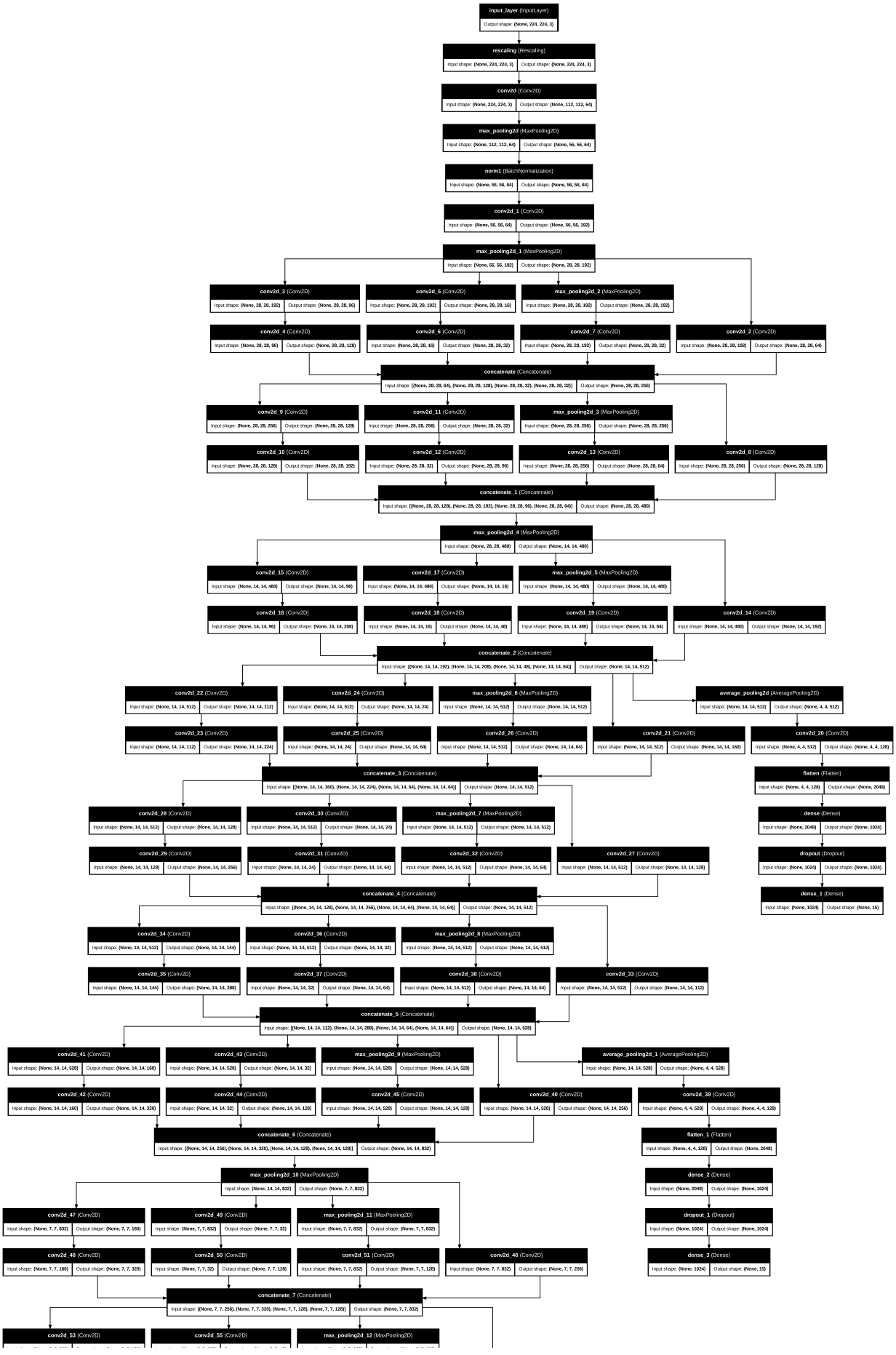
Trainable params: 10,345,373 (39.46 MB)

Non-trainable params: 128 (512.00 B)

In [ ]:
```
from tensorflow.keras.utils import plot_model

# Save the model plot to a file with higher resolution
plot_model(
    googlenet,
    to_file="googlenet_model.png",
    show_shapes=True,
    show_layer_names=True,
    dpi=300
)
```

Out[ ]:


input_layer (InputLayer)
Output shape: (None, 224, 224, 3)
rescaling (Rescaling)
Input shape: (None, 224, 224, 3) | Output shape: (None, 224, 224, 3)
conv2d (Conv2D)
Input shape: (None, 224, 224, 3) | Output shape: (None, 112, 112, 64)
max_pooling2d (MaxPooling2D)
Input shape: (None, 112, 112, 64) | Output shape: (None, 56, 56, 64)
norm1 (BatchNormalization)
Input shape: (None, 56, 56, 64) | Output shape: (None, 56, 56, 64)
conv2d_1 (Conv2D)
Input shape: (None, 56, 56, 64) | Output shape: (None, 56, 56, 192)
max_pooling2d_1 (MaxPooling2D)
Input shape: (None, 56, 56, 192) | Output shape: (None, 28, 28, 192)
conv2d_3 (Conv2D)
Input shape: (None, 28, 28, 192) | Output shape: (None, 28, 28, 96)
conv2d_5 (Conv2D)
Input shape: (None, 28, 28, 192) | Output shape: (None, 28, 28, 16)
max_pooling2d_2 (MaxPooling2D)
Input shape: (None, 28, 28, 192) | Output shape: (None, 28, 28, 192)
conv2d_4 (Conv2D)
Input shape: (None, 28, 28, 96) | Output shape: (None, 28, 28, 128)
conv2d_6 (Conv2D)
Input shape: (None, 28, 28, 16) | Output shape: (None, 28, 28, 32)
conv2d_7 (Conv2D)
Input shape: (None, 28, 28, 192) | Output shape: (None, 28, 28, 32)
conv2d_2 (Conv2D)
Input shape: (None, 28, 28, 192) | Output shape: (None, 28, 28, 64)
concatenate (Concatenate)
Input shape: [(None, 28, 28, 64), (None, 28, 28, 128), (None, 28, 28, 32), (None, 28, 28, 32)] | Output shape: (None, 28, 28, 256)
conv2d_9 (Conv2D)
Input shape: (None, 28, 28, 256) | Output shape: (None, 28, 28, 128)
conv2d_11 (Conv2D)
Input shape: (None, 28, 28, 256) | Output shape: (None, 28, 28, 32)
max_pooling2d_3 (MaxPooling2D)
Input shape: (None, 28, 28, 256) | Output shape: (None, 28, 28, 256)
conv2d_10 (Conv2D)
Input shape: (None, 28, 28, 128) | Output shape: (None, 28, 28, 192)
conv2d_12 (Conv2D)
Input shape: (None, 28, 28, 32) | Output shape: (None, 28, 28, 96)
conv2d_13 (Conv2D)
Input shape: (None, 28, 28, 256) | Output shape: (None, 28, 28, 64)
conv2d_8 (Conv2D)
Input shape: (None, 28, 28, 256) | Output shape: (None, 28, 28, 128)
concatenate_1 (Concatenate)
Input shape: [(None, 28, 28, 128), (None, 28, 28, 192), (None, 28, 28, 96), (None, 28, 28, 64)] | Output shape: (None, 28, 28, 480)
max_pooling2d_4 (MaxPooling2D)
Input shape: (None, 28, 28, 480) | Output shape: (None, 14, 14, 480)
conv2d_15 (Conv2D)
Input shape: (None, 14, 14, 480) | Output shape: (None, 14, 14, 96)
conv2d_17 (Conv2D)
Input shape: (None, 14, 14, 480) | Output shape: (None, 14, 14, 16)
max_pooling2d_5 (MaxPooling2D)
Input shape: (None, 14, 14, 480) | Output shape: (None, 14, 14, 480)
conv2d_16 (Conv2D)
Input shape: (None, 14, 14, 96) | Output shape: (None, 14, 14, 208)
conv2d_18 (Conv2D)
Input shape: (None, 14, 14, 16) | Output shape: (None, 14, 14, 48)
conv2d_19 (Conv2D)
Input shape: (None, 14, 14, 480) | Output shape: (None, 14, 14, 64)
conv2d_14 (Conv2D)
Input shape: (None, 14, 14, 480) | Output shape: (None, 14, 14, 192)
concatenate_2 (Concatenate)
Input shape: [(None, 14, 14, 192), (None, 14, 14, 208), (None, 14, 14, 48), (None, 14, 14, 64)] | Output shape: (None, 14, 14, 512)
conv2d_22 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 112)
conv2d_24 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 24)
max_pooling2d_6 (MaxPooling2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 512)
average_pooling2d (AveragePooling2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 4, 4, 512)
conv2d_23 (Conv2D)
Input shape: (None, 14, 14, 112) | Output shape: (None, 14, 14, 224)
conv2d_25 (Conv2D)
Input shape: (None, 14, 14, 24) | Output shape: (None, 14, 14, 64)
conv2d_26 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 64)
conv2d_21 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 160)
conv2d_20 (Conv2D)
Input shape: (None, 4, 4, 512) | Output shape: (None, 4, 4, 128)
concatenate_3 (Concatenate)
Input shape: [(None, 14, 14, 160), (None, 14, 14, 224), (None, 14, 14, 64), (None, 14, 14, 64)] | Output shape: (None, 14, 14, 512)
flatten (Flatten)
Input shape: (None, 4, 4, 128) | Output shape: (None, 2048)
conv2d_28 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 128)
conv2d_30 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 24)
max_pooling2d_7 (MaxPooling2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 512)
dense (Dense)
Input shape: (None, 2048) | Output shape: (None, 1024)
conv2d_29 (Conv2D)
Input shape: (None, 14, 14, 128) | Output shape: (None, 14, 14, 256)
conv2d_31 (Conv2D)
Input shape: (None, 14, 14, 24) | Output shape: (None, 14, 14, 64)
conv2d_32 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 64)
conv2d_27 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 128)
dropout (Dropout)
Input shape: (None, 1024) | Output shape: (None, 1024)
concatenate_4 (Concatenate)
Input shape: [(None, 14, 14, 128), (None, 14, 14, 256), (None, 14, 14, 64), (None, 14, 14, 64)] | Output shape: (None, 14, 14, 512)
dense_1 (Dense)
Input shape: (None, 1024) | Output shape: (None, 15)
conv2d_34 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 144)
conv2d_36 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 32)
max_pooling2d_8 (MaxPooling2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 512)
conv2d_35 (Conv2D)
Input shape: (None, 14, 14, 144) | Output shape: (None, 14, 14, 288)
conv2d_37 (Conv2D)
Input shape: (None, 14, 14, 32) | Output shape: (None, 14, 14, 64)
conv2d_38 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 64)
conv2d_33 (Conv2D)
Input shape: (None, 14, 14, 512) | Output shape: (None, 14, 14, 112)
concatenate_5 (Concatenate)
Input shape: [(None, 14, 14, 112), (None, 14, 14, 288), (None, 14, 14, 64), (None, 14, 14, 64)] | Output shape: (None, 14, 14, 528)
conv2d_41 (Conv2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 14, 14, 160)
conv2d_43 (Conv2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 14, 14, 32)
max_pooling2d_9 (MaxPooling2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 14, 14, 528)
average_pooling2d_1 (AveragePooling2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 4, 4, 528)
conv2d_42 (Conv2D)
Input shape: (None, 14, 14, 160) | Output shape: (None, 14, 14, 320)
conv2d_44 (Conv2D)
Input shape: (None, 14, 14, 32) | Output shape: (None, 14, 14, 128)
conv2d_45 (Conv2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 14, 14, 128)
conv2d_40 (Conv2D)
Input shape: (None, 14, 14, 528) | Output shape: (None, 14, 14, 256)
conv2d_39 (Conv2D)
Input shape: (None, 4, 4, 528) | Output shape: (None, 4, 4, 128)
concatenate_6 (Concatenate)
Input shape: [(None, 14, 14, 256), (None, 14, 14, 320), (None, 14, 14, 128), (None, 14, 14, 128)] | Output shape: (None, 14, 14, 832)
flatten_1 (Flatten)
Input shape: (None, 4, 4, 128) | Output shape: (None, 2048)
max_pooling2d_10 (MaxPooling2D)
Input shape: (None, 14, 14, 832) | Output shape: (None, 7, 7, 832)
dense_2 (Dense)
Input shape: (None, 2048) | Output shape: (None, 1024)
conv2d_47 (Conv2D)
Input shape: (None, 7, 7, 832) | Output shape: (None, 7, 7, 160)
conv2d_49 (Conv2D)
Input shape: (None, 7, 7, 832) | Output shape: (None, 7, 7, 32)
max_pooling2d_11 (MaxPooling2D)
Input shape: (None, 7, 7, 832) | Output shape: (None, 7, 7, 832)
dropout_1 (Dropout)
Input shape: (None, 1024) | Output shape: (None, 1024)
conv2d_48 (Conv2D)
Input shape: (None, 7, 7, 160) | Output shape: (None, 7, 7, 320)
conv2d_50 (Conv2D)
Input shape: (None, 7, 7, 32) | Output shape: (None, 7, 7, 128)
conv2d_51 (Conv2D)
Input shape: (None, 7, 7, 832) | Output shape: (None, 7, 7, 128)
conv2d_46 (Conv2D)
Input shape: (None, 7, 7, 832) | Output shape: (None, 7, 7, 256)
dense_3 (Dense)
Input shape: (None, 1024) | Output shape: (None, 15)
concatenate_7 (Concatenate)
Input shape: [(None, 7, 7, 256), (None, 7, 7, 320), (None, 7, 7, 128), (None, 7, 7, 128)] | Output shape: (None, 7, 7, 832)
conv2d_53 (Conv2D)
conv2d_55 (Conv2D)
max_pooling2d_12 (MaxPooling2D)

In [ ]:
```
googlenet.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss=['sparse_categorical_crossentropy', 'sparse_categorical_crossentropy'
    loss_weights=[1.0, 0.3, 0.3],
    metrics=['accuracy', 'accuracy', 'accuracy']
)
```

In [ ]:
```
history = googlenet.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=50
)
```

```
Epoch 1/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 100s 301ms/step - dense_1_accuracy: 0.0762 - dens
e_1_loss: 2.7603 - dense_3_accuracy: 0.0702 - dense_3_loss: 2.7070 - dense_4_ac
curacy: 0.0729 - dense_4_loss: 2.7052 - loss: 4.3454 - val_dense_1_accuracy:
0.1461 - val_dense_1_loss: 2.6906 - val_dense_3_accuracy: 0.1208 - val_dens
e_3_loss: 2.6981 - val_dense_4_accuracy: 0.0942 - val_dense_4_loss: 2.6991 - va
l_loss: 4.3157
Epoch 2/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 23s 124ms/step - dense_1_accuracy: 0.1253 - dens
e_1_loss: 2.6429 - dense_3_accuracy: 0.1147 - dense_3_loss: 2.6606 - dense_4_ac
curacy: 0.1115 - dense_4_loss: 2.6628 - loss: 4.2538 - val_dense_1_accuracy:
0.1836 - val_dense_1_loss: 2.6096 - val_dense_3_accuracy: 0.1604 - val_dens
e_3_loss: 2.6220 - val_dense_4_accuracy: 0.1693 - val_dense_4_loss: 2.6184 - va
l_loss: 4.1880
Epoch 3/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 40s 120ms/step - dense_1_accuracy: 0.1628 - dens
e_1_loss: 2.5599 - dense_3_accuracy: 0.1542 - dense_3_loss: 2.5782 - dense_4_ac
curacy: 0.1513 - dense_4_loss: 2.5566 - loss: 4.0981 - val_dense_1_accuracy:
0.2341 - val_dense_1_loss: 2.4654 - val_dense_3_accuracy: 0.2157 - val_dens
e_3_loss: 2.4737 - val_dense_4_accuracy: 0.2102 - val_dense_4_loss: 2.4573 - va
l_loss: 3.9387
Epoch 4/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 25s 134ms/step - dense_1_accuracy: 0.2024 - dens
e_1_loss: 2.4700 - dense_3_accuracy: 0.2007 - dense_3_loss: 2.4789 - dense_4_ac
curacy: 0.2079 - dense_4_loss: 2.4384 - loss: 3.9230 - val_dense_1_accuracy:
0.2614 - val_dense_1_loss: 2.3319 - val_dense_3_accuracy: 0.2532 - val_dens
e_3_loss: 2.3131 - val_dense_4_accuracy: 0.2676 - val_dense_4_loss: 2.2866 - va
l_loss: 3.6796
Epoch 5/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 39s 122ms/step - dense_1_accuracy: 0.2202 - dens
e_1_loss: 2.4171 - dense_3_accuracy: 0.2220 - dense_3_loss: 2.4005 - dense_4_ac
curacy: 0.2525 - dense_4_loss: 2.3341 - loss: 3.7794 - val_dense_1_accuracy:
0.2840 - val_dense_1_loss: 2.2554 - val_dense_3_accuracy: 0.3003 - val_dens
e_3_loss: 2.2263 - val_dense_4_accuracy: 0.3113 - val_dense_4_loss: 2.1777 - va
l_loss: 3.5223
Epoch 6/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 43s 133ms/step - dense_1_accuracy: 0.2618 - dens
e_1_loss: 2.3349 - dense_3_accuracy: 0.2646 - dense_3_loss: 2.2992 - dense_4_ac
curacy: 0.2877 - dense_4_loss: 2.2354 - loss: 3.6256 - val_dense_1_accuracy:
0.3085 - val_dense_1_loss: 2.2030 - val_dense_3_accuracy: 0.3181 - val_dens
e_3_loss: 2.1617 - val_dense_4_accuracy: 0.3297 - val_dense_4_loss: 2.1169 - va
l_loss: 3.4243
Epoch 7/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 23s 126ms/step - dense_1_accuracy: 0.2687 - dens
e_1_loss: 2.2933 - dense_3_accuracy: 0.2893 - dense_3_loss: 2.2425 - dense_4_ac
curacy: 0.3094 - dense_4_loss: 2.1541 - loss: 3.5149 - val_dense_1_accuracy:
0.3365 - val_dense_1_loss: 2.1461 - val_dense_3_accuracy: 0.3543 - val_dens
e_3_loss: 2.0800 - val_dense_4_accuracy: 0.3625 - val_dense_4_loss: 2.0341 - va
l_loss: 3.3040
Epoch 8/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 40s 123ms/step - dense_1_accuracy: 0.2801 - dens
e_1_loss: 2.2402 - dense_3_accuracy: 0.3055 - dense_3_loss: 2.1885 - dense_4_ac
curacy: 0.3379 - dense_4_loss: 2.0914 - loss: 3.4200 - val_dense_1_accuracy:
0.3454 - val_dense_1_loss: 2.1005 - val_dense_3_accuracy: 0.3625 - val_dens
```

e_3_loss: 2.0226 - val_dense_4_accuracy: 0.3590 - val_dense_4_loss: 1.9948 - va
l_loss: 3.2332
Epoch 9/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 126ms/step - dense_1_accuracy: 0.3061 - dens
e_1_loss: 2.2005 - dense_3_accuracy: 0.3342 - dense_3_loss: 2.1186 - dense_4_ac
curacy: 0.3444 - dense_4_loss: 2.0370 - loss: 3.3327 - val_dense_1_accuracy:
0.3611 - val_dense_1_loss: 2.0636 - val_dense_3_accuracy: 0.3891 - val_dens
e_3_loss: 1.9766 - val_dense_4_accuracy: 0.3857 - val_dense_4_loss: 1.9580 - va
l_loss: 3.1692
Epoch 10/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **44s** 141ms/step - dense_1_accuracy: 0.3110 - dens
e_1_loss: 2.1470 - dense_3_accuracy: 0.3466 - dense_3_loss: 2.0609 - dense_4_ac
curacy: 0.3704 - dense_4_loss: 1.9836 - loss: 3.2460 - val_dense_1_accuracy:
0.3727 - val_dense_1_loss: 2.0351 - val_dense_3_accuracy: 0.3898 - val_dens
e_3_loss: 1.9426 - val_dense_4_accuracy: 0.3823 - val_dense_4_loss: 1.9222 - va
l_loss: 3.1151
Epoch 11/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 124ms/step - dense_1_accuracy: 0.3392 - dens
e_1_loss: 2.1091 - dense_3_accuracy: 0.3626 - dense_3_loss: 2.0080 - dense_4_ac
curacy: 0.3868 - dense_4_loss: 1.9296 - loss: 3.1648 - val_dense_1_accuracy:
0.3857 - val_dense_1_loss: 2.0074 - val_dense_3_accuracy: 0.4055 - val_dens
e_3_loss: 1.8963 - val_dense_4_accuracy: 0.4177 - val_dense_4_loss: 1.8725 - va
l_loss: 3.0417
Epoch 12/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 137ms/step - dense_1_accuracy: 0.3409 - dens
e_1_loss: 2.0843 - dense_3_accuracy: 0.3861 - dense_3_loss: 1.9696 - dense_4_ac
curacy: 0.4003 - dense_4_loss: 1.8852 - loss: 3.1014 - val_dense_1_accuracy:
0.3870 - val_dense_1_loss: 1.9786 - val_dense_3_accuracy: 0.4041 - val_dens
e_3_loss: 1.8651 - val_dense_4_accuracy: 0.4014 - val_dense_4_loss: 1.8591 - va
l_loss: 3.0116
Epoch 13/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **41s** 135ms/step - dense_1_accuracy: 0.3553 - dens
e_1_loss: 2.0513 - dense_3_accuracy: 0.3859 - dense_3_loss: 1.9400 - dense_4_ac
curacy: 0.3985 - dense_4_loss: 1.8693 - loss: 3.0666 - val_dense_1_accuracy:
0.4020 - val_dense_1_loss: 1.9566 - val_dense_3_accuracy: 0.4150 - val_dens
e_3_loss: 1.8440 - val_dense_4_accuracy: 0.4253 - val_dense_4_loss: 1.8319 - va
l_loss: 2.9727
Epoch 14/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 127ms/step - dense_1_accuracy: 0.3604 - dens
e_1_loss: 2.0372 - dense_3_accuracy: 0.3900 - dense_3_loss: 1.9130 - dense_4_ac
curacy: 0.4138 - dense_4_loss: 1.8405 - loss: 3.0255 - val_dense_1_accuracy:
0.4055 - val_dense_1_loss: 1.9394 - val_dense_3_accuracy: 0.4130 - val_dens
e_3_loss: 1.8238 - val_dense_4_accuracy: 0.4130 - val_dense_4_loss: 1.8091 - va
l_loss: 2.9379
Epoch 15/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 125ms/step - dense_1_accuracy: 0.3581 - dens
e_1_loss: 2.0181 - dense_3_accuracy: 0.4155 - dense_3_loss: 1.8843 - dense_4_ac
curacy: 0.4162 - dense_4_loss: 1.8033 - loss: 2.9740 - val_dense_1_accuracy:
0.4020 - val_dense_1_loss: 1.9244 - val_dense_3_accuracy: 0.4328 - val_dens
e_3_loss: 1.7992 - val_dense_4_accuracy: 0.4280 - val_dense_4_loss: 1.7936 - va
l_loss: 2.9116
Epoch 16/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **43s** 136ms/step - dense_1_accuracy: 0.3803 - dens
e_1_loss: 1.9673 - dense_3_accuracy: 0.4209 - dense_3_loss: 1.8196 - dense_4_ac

curacy: 0.4280 - dense_4_loss: 1.7685 - loss: 2.9046 - val_dense_1_accuracy:
0.4075 - val_dense_1_loss: 1.9155 - val_dense_3_accuracy: 0.4184 - val_dens
e_3_loss: 1.8062 - val_dense_4_accuracy: 0.4102 - val_dense_4_loss: 1.8167 - va
l_loss: 2.9354
Epoch 17/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **41s** 137ms/step - dense_1_accuracy: 0.3869 - dens
e_1_loss: 1.9644 - dense_3_accuracy: 0.4328 - dense_3_loss: 1.8105 - dense_4_ac
curacy: 0.4441 - dense_4_loss: 1.7438 - loss: 2.8763 - val_dense_1_accuracy:
0.4171 - val_dense_1_loss: 1.8875 - val_dense_3_accuracy: 0.4328 - val_dens
e_3_loss: 1.7729 - val_dense_4_accuracy: 0.4300 - val_dense_4_loss: 1.7644 - va
l_loss: 2.8656
Epoch 18/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **39s** 127ms/step - dense_1_accuracy: 0.3898 - dens
e_1_loss: 1.9413 - dense_3_accuracy: 0.4374 - dense_3_loss: 1.7975 - dense_4_ac
curacy: 0.4557 - dense_4_loss: 1.7024 - loss: 2.8241 - val_dense_1_accuracy:
0.4239 - val_dense_1_loss: 1.8735 - val_dense_3_accuracy: 0.4519 - val_dens
e_3_loss: 1.7528 - val_dense_4_accuracy: 0.4512 - val_dense_4_loss: 1.7516 - va
l_loss: 2.8410
Epoch 19/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **43s** 136ms/step - dense_1_accuracy: 0.3959 - dens
e_1_loss: 1.9273 - dense_3_accuracy: 0.4383 - dense_3_loss: 1.7642 - dense_4_ac
curacy: 0.4662 - dense_4_loss: 1.6830 - loss: 2.7904 - val_dense_1_accuracy:
0.4253 - val_dense_1_loss: 1.8585 - val_dense_3_accuracy: 0.4608 - val_dens
e_3_loss: 1.7319 - val_dense_4_accuracy: 0.4560 - val_dense_4_loss: 1.7241 - va
l_loss: 2.8011
Epoch 20/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **39s** 128ms/step - dense_1_accuracy: 0.3946 - dens
e_1_loss: 1.9031 - dense_3_accuracy: 0.4513 - dense_3_loss: 1.7382 - dense_4_ac
curacy: 0.4583 - dense_4_loss: 1.6767 - loss: 2.7691 - val_dense_1_accuracy:
0.4259 - val_dense_1_loss: 1.8470 - val_dense_3_accuracy: 0.4573 - val_dens
e_3_loss: 1.7283 - val_dense_4_accuracy: 0.4621 - val_dense_4_loss: 1.7250 - va
l_loss: 2.7986
Epoch 21/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 124ms/step - dense_1_accuracy: 0.4144 - dens
e_1_loss: 1.8599 - dense_3_accuracy: 0.4661 - dense_3_loss: 1.7034 - dense_4_ac
curacy: 0.4799 - dense_4_loss: 1.6317 - loss: 2.7007 - val_dense_1_accuracy:
0.4321 - val_dense_1_loss: 1.8322 - val_dense_3_accuracy: 0.4676 - val_dens
e_3_loss: 1.7184 - val_dense_4_accuracy: 0.4621 - val_dense_4_loss: 1.7129 - va
l_loss: 2.7781
Epoch 22/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 137ms/step - dense_1_accuracy: 0.4160 - dens
e_1_loss: 1.8695 - dense_3_accuracy: 0.4620 - dense_3_loss: 1.6972 - dense_4_ac
curacy: 0.4883 - dense_4_loss: 1.6111 - loss: 2.6812 - val_dense_1_accuracy:
0.4444 - val_dense_1_loss: 1.8216 - val_dense_3_accuracy: 0.4669 - val_dens
e_3_loss: 1.7113 - val_dense_4_accuracy: 0.4628 - val_dense_4_loss: 1.7273 - va
l_loss: 2.7849
Epoch 23/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 124ms/step - dense_1_accuracy: 0.4258 - dens
e_1_loss: 1.8292 - dense_3_accuracy: 0.4809 - dense_3_loss: 1.6726 - dense_4_ac
curacy: 0.4912 - dense_4_loss: 1.5889 - loss: 2.6394 - val_dense_1_accuracy:
0.4396 - val_dense_1_loss: 1.8051 - val_dense_3_accuracy: 0.4676 - val_dens
e_3_loss: 1.6770 - val_dense_4_accuracy: 0.4867 - val_dense_4_loss: 1.6606 - va
l_loss: 2.7073
Epoch 24/50

```
184/184 ━━━━━━━━━━━━━━━━━━━━ 23s 125ms/step - dense_1_accuracy: 0.4332 - dens
e_1_loss: 1.8160 - dense_3_accuracy: 0.4749 - dense_3_loss: 1.6398 - dense_4_ac
curacy: 0.4937 - dense_4_loss: 1.5661 - loss: 2.6029 - val_dense_1_accuracy:
0.4410 - val_dense_1_loss: 1.7953 - val_dense_3_accuracy: 0.4744 - val_dens
e_3_loss: 1.6817 - val_dense_4_accuracy: 0.4730 - val_dense_4_loss: 1.6847 - va
l_loss: 2.7303
Epoch 25/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 25s 138ms/step - dense_1_accuracy: 0.4310 - dens
e_1_loss: 1.8103 - dense_3_accuracy: 0.4848 - dense_3_loss: 1.6302 - dense_4_ac
curacy: 0.5055 - dense_4_loss: 1.5462 - loss: 2.5783 - val_dense_1_accuracy:
0.4430 - val_dense_1_loss: 1.7879 - val_dense_3_accuracy: 0.4724 - val_dens
e_3_loss: 1.6862 - val_dense_4_accuracy: 0.4778 - val_dense_4_loss: 1.6764 - va
l_loss: 2.7192
Epoch 26/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 41s 136ms/step - dense_1_accuracy: 0.4368 - dens
e_1_loss: 1.7982 - dense_3_accuracy: 0.4827 - dense_3_loss: 1.6326 - dense_4_ac
curacy: 0.5124 - dense_4_loss: 1.5522 - loss: 2.5814 - val_dense_1_accuracy:
0.4512 - val_dense_1_loss: 1.7750 - val_dense_3_accuracy: 0.4881 - val_dens
e_3_loss: 1.6393 - val_dense_4_accuracy: 0.4969 - val_dense_4_loss: 1.6153 - va
l_loss: 2.6419
Epoch 27/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 39s 124ms/step - dense_1_accuracy: 0.4353 - dens
e_1_loss: 1.7993 - dense_3_accuracy: 0.4960 - dense_3_loss: 1.5982 - dense_4_ac
curacy: 0.5272 - dense_4_loss: 1.5190 - loss: 2.5382 - val_dense_1_accuracy:
0.4553 - val_dense_1_loss: 1.7698 - val_dense_3_accuracy: 0.4922 - val_dens
e_3_loss: 1.6400 - val_dense_4_accuracy: 0.4956 - val_dense_4_loss: 1.6274 - va
l_loss: 2.6503
Epoch 28/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 23s 127ms/step - dense_1_accuracy: 0.4447 - dens
e_1_loss: 1.7697 - dense_3_accuracy: 0.5008 - dense_3_loss: 1.5905 - dense_4_ac
curacy: 0.5243 - dense_4_loss: 1.4948 - loss: 2.5029 - val_dense_1_accuracy:
0.4553 - val_dense_1_loss: 1.7560 - val_dense_3_accuracy: 0.4887 - val_dens
e_3_loss: 1.6534 - val_dense_4_accuracy: 0.4942 - val_dense_4_loss: 1.6683 - va
l_loss: 2.6942
Epoch 29/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 41s 124ms/step - dense_1_accuracy: 0.4406 - dens
e_1_loss: 1.7527 - dense_3_accuracy: 0.5137 - dense_3_loss: 1.5662 - dense_4_ac
curacy: 0.5292 - dense_4_loss: 1.4863 - loss: 2.4819 - val_dense_1_accuracy:
0.4655 - val_dense_1_loss: 1.7524 - val_dense_3_accuracy: 0.4922 - val_dens
e_3_loss: 1.6629 - val_dense_4_accuracy: 0.4812 - val_dense_4_loss: 1.6785 - va
l_loss: 2.7025
Epoch 30/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 25s 137ms/step - dense_1_accuracy: 0.4492 - dens
e_1_loss: 1.7474 - dense_3_accuracy: 0.5065 - dense_3_loss: 1.5560 - dense_4_ac
curacy: 0.5367 - dense_4_loss: 1.4577 - loss: 2.4487 - val_dense_1_accuracy:
0.4580 - val_dense_1_loss: 1.7445 - val_dense_3_accuracy: 0.4908 - val_dens
e_3_loss: 1.6533 - val_dense_4_accuracy: 0.4949 - val_dense_4_loss: 1.6582 - va
l_loss: 2.6785
Epoch 31/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 39s 124ms/step - dense_1_accuracy: 0.4572 - dens
e_1_loss: 1.7216 - dense_3_accuracy: 0.5298 - dense_3_loss: 1.5131 - dense_4_ac
curacy: 0.5443 - dense_4_loss: 1.4279 - loss: 2.3982 - val_dense_1_accuracy:
0.4614 - val_dense_1_loss: 1.7349 - val_dense_3_accuracy: 0.4778 - val_dens
e_3_loss: 1.6839 - val_dense_4_accuracy: 0.4744 - val_dense_4_loss: 1.7131 - va
```

l_loss: 2.7376
Epoch 32/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **43s** 136ms/step - dense_1_accuracy: 0.4536 - dens
e_1_loss: 1.7164 - dense_3_accuracy: 0.5159 - dense_3_loss: 1.5388 - dense_4_ac
curacy: 0.5404 - dense_4_loss: 1.4486 - loss: 2.4251 - val_dense_1_accuracy:
0.4648 - val_dense_1_loss: 1.7250 - val_dense_3_accuracy: 0.4990 - val_dens
e_3_loss: 1.6248 - val_dense_4_accuracy: 0.5024 - val_dense_4_loss: 1.6312 - va
l_loss: 2.6370
Epoch 33/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 125ms/step - dense_1_accuracy: 0.4746 - dens
e_1_loss: 1.6958 - dense_3_accuracy: 0.5286 - dense_3_loss: 1.5006 - dense_4_ac
curacy: 0.5584 - dense_4_loss: 1.4046 - loss: 2.3635 - val_dense_1_accuracy:
0.4717 - val_dense_1_loss: 1.7194 - val_dense_3_accuracy: 0.5051 - val_dens
e_3_loss: 1.5897 - val_dense_4_accuracy: 0.5188 - val_dense_4_loss: 1.5690 - va
l_loss: 2.5590
Epoch 34/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **24s** 128ms/step - dense_1_accuracy: 0.4730 - dens
e_1_loss: 1.6934 - dense_3_accuracy: 0.5276 - dense_3_loss: 1.4790 - dense_4_ac
curacy: 0.5557 - dense_4_loss: 1.3787 - loss: 2.3304 - val_dense_1_accuracy:
0.4765 - val_dense_1_loss: 1.7088 - val_dense_3_accuracy: 0.5160 - val_dens
e_3_loss: 1.5877 - val_dense_4_accuracy: 0.5167 - val_dense_4_loss: 1.5819 - va
l_loss: 2.5699
Epoch 35/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **24s** 130ms/step - dense_1_accuracy: 0.4735 - dens
e_1_loss: 1.6769 - dense_3_accuracy: 0.5333 - dense_3_loss: 1.4753 - dense_4_ac
curacy: 0.5659 - dense_4_loss: 1.3697 - loss: 2.3154 - val_dense_1_accuracy:
0.4751 - val_dense_1_loss: 1.7053 - val_dense_3_accuracy: 0.5078 - val_dens
e_3_loss: 1.6090 - val_dense_4_accuracy: 0.5024 - val_dense_4_loss: 1.6048 - va
l_loss: 2.6002
Epoch 36/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 137ms/step - dense_1_accuracy: 0.4782 - dens
e_1_loss: 1.6770 - dense_3_accuracy: 0.5304 - dense_3_loss: 1.4738 - dense_4_ac
curacy: 0.5615 - dense_4_loss: 1.3767 - loss: 2.3219 - val_dense_1_accuracy:
0.4771 - val_dense_1_loss: 1.7052 - val_dense_3_accuracy: 0.5058 - val_dens
e_3_loss: 1.5868 - val_dense_4_accuracy: 0.5140 - val_dense_4_loss: 1.5690 - va
l_loss: 2.5540
Epoch 37/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 134ms/step - dense_1_accuracy: 0.4792 - dens
e_1_loss: 1.6533 - dense_3_accuracy: 0.5461 - dense_3_loss: 1.4410 - dense_4_ac
curacy: 0.5635 - dense_4_loss: 1.3560 - loss: 2.2843 - val_dense_1_accuracy:
0.4833 - val_dense_1_loss: 1.6899 - val_dense_3_accuracy: 0.5126 - val_dens
e_3_loss: 1.5783 - val_dense_4_accuracy: 0.5174 - val_dense_4_loss: 1.5774 - va
l_loss: 2.5609
Epoch 38/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 136ms/step - dense_1_accuracy: 0.4854 - dens
e_1_loss: 1.6459 - dense_3_accuracy: 0.5595 - dense_3_loss: 1.4231 - dense_4_ac
curacy: 0.5741 - dense_4_loss: 1.3344 - loss: 2.2552 - val_dense_1_accuracy:
0.4908 - val_dense_1_loss: 1.6857 - val_dense_3_accuracy: 0.5195 - val_dens
e_3_loss: 1.5563 - val_dense_4_accuracy: 0.5242 - val_dense_4_loss: 1.5296 - va
l_loss: 2.5021
Epoch 39/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 137ms/step - dense_1_accuracy: 0.4844 - dens
e_1_loss: 1.6363 - dense_3_accuracy: 0.5603 - dense_3_loss: 1.3950 - dense_4_ac
curacy: 0.5892 - dense_4_loss: 1.2990 - loss: 2.2084 - val_dense_1_accuracy:

0.4840 - val_dense_1_loss: 1.6775 - val_dense_3_accuracy: 0.5195 - val_dens
e_3_loss: 1.5570 - val_dense_4_accuracy: 0.5167 - val_dense_4_loss: 1.5388 - va
l_loss: 2.5093
Epoch 40/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **41s** 138ms/step - dense_1_accuracy: 0.4852 - dens
e_1_loss: 1.6411 - dense_3_accuracy: 0.5529 - dense_3_loss: 1.4166 - dense_4_ac
curacy: 0.5749 - dense_4_loss: 1.3001 - loss: 2.2174 - val_dense_1_accuracy:
0.4949 - val_dense_1_loss: 1.6711 - val_dense_3_accuracy: 0.5235 - val_dens
e_3_loss: 1.5582 - val_dense_4_accuracy: 0.5297 - val_dense_4_loss: 1.5489 - va
l_loss: 2.5176
Epoch 41/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **26s** 138ms/step - dense_1_accuracy: 0.4876 - dens
e_1_loss: 1.6148 - dense_3_accuracy: 0.5592 - dense_3_loss: 1.3890 - dense_4_ac
curacy: 0.5998 - dense_4_loss: 1.2635 - loss: 2.1646 - val_dense_1_accuracy:
0.4833 - val_dense_1_loss: 1.6679 - val_dense_3_accuracy: 0.5167 - val_dens
e_3_loss: 1.5954 - val_dense_4_accuracy: 0.5147 - val_dense_4_loss: 1.6057 - va
l_loss: 2.5885
Epoch 42/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **25s** 138ms/step - dense_1_accuracy: 0.4943 - dens
e_1_loss: 1.5997 - dense_3_accuracy: 0.5644 - dense_3_loss: 1.3680 - dense_4_ac
curacy: 0.5978 - dense_4_loss: 1.2588 - loss: 2.1492 - val_dense_1_accuracy:
0.4928 - val_dense_1_loss: 1.6643 - val_dense_3_accuracy: 0.5311 - val_dens
e_3_loss: 1.5404 - val_dense_4_accuracy: 0.5365 - val_dense_4_loss: 1.5106 - va
l_loss: 2.4711
Epoch 43/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **41s** 136ms/step - dense_1_accuracy: 0.5021 - dens
e_1_loss: 1.5861 - dense_3_accuracy: 0.5616 - dense_3_loss: 1.3652 - dense_4_ac
curacy: 0.6085 - dense_4_loss: 1.2309 - loss: 2.1162 - val_dense_1_accuracy:
0.4942 - val_dense_1_loss: 1.6537 - val_dense_3_accuracy: 0.5352 - val_dens
e_3_loss: 1.5487 - val_dense_4_accuracy: 0.5304 - val_dense_4_loss: 1.5365 - va
l_loss: 2.4976
Epoch 44/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **24s** 128ms/step - dense_1_accuracy: 0.4975 - dens
e_1_loss: 1.5885 - dense_3_accuracy: 0.5734 - dense_3_loss: 1.3348 - dense_4_ac
curacy: 0.6070 - dense_4_loss: 1.2211 - loss: 2.0982 - val_dense_1_accuracy:
0.4935 - val_dense_1_loss: 1.6508 - val_dense_3_accuracy: 0.5208 - val_dens
e_3_loss: 1.5717 - val_dense_4_accuracy: 0.5229 - val_dense_4_loss: 1.5653 - va
l_loss: 2.5316
Epoch 45/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **23s** 126ms/step - dense_1_accuracy: 0.5003 - dens
e_1_loss: 1.5689 - dense_3_accuracy: 0.5898 - dense_3_loss: 1.3130 - dense_4_ac
curacy: 0.6127 - dense_4_loss: 1.2026 - loss: 2.0671 - val_dense_1_accuracy:
0.5017 - val_dense_1_loss: 1.6417 - val_dense_3_accuracy: 0.5358 - val_dens
e_3_loss: 1.5305 - val_dense_4_accuracy: 0.5331 - val_dense_4_loss: 1.5286 - va
l_loss: 2.4817
Epoch 46/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **43s** 136ms/step - dense_1_accuracy: 0.5067 - dens
e_1_loss: 1.5807 - dense_3_accuracy: 0.5915 - dense_3_loss: 1.3119 - dense_4_ac
curacy: 0.6163 - dense_4_loss: 1.2055 - loss: 2.0733 - val_dense_1_accuracy:
0.4942 - val_dense_1_loss: 1.6383 - val_dense_3_accuracy: 0.5338 - val_dens
e_3_loss: 1.5128 - val_dense_4_accuracy: 0.5474 - val_dense_4_loss: 1.4810 - va
l_loss: 2.4274
Epoch 47/50
184/184 ━━━━━━━━━━━━━━━━━━━━ **26s** 138ms/step - dense_1_accuracy: 0.5254 - dens

```
e_1_loss: 1.5312 - dense_3_accuracy: 0.5972 - dense_3_loss: 1.2748 - dense_4_ac
curacy: 0.6316 - dense_4_loss: 1.1487 - loss: 1.9905 - val_dense_1_accuracy:
0.4997 - val_dense_1_loss: 1.6354 - val_dense_3_accuracy: 0.5324 - val_dens
e_3_loss: 1.5285 - val_dense_4_accuracy: 0.5358 - val_dense_4_loss: 1.5045 - va
l_loss: 2.4571
Epoch 48/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 23s 126ms/step - dense_1_accuracy: 0.5103 - dens
e_1_loss: 1.5491 - dense_3_accuracy: 0.5970 - dense_3_loss: 1.2764 - dense_4_ac
curacy: 0.6389 - dense_4_loss: 1.1524 - loss: 2.0000 - val_dense_1_accuracy:
0.5017 - val_dense_1_loss: 1.6250 - val_dense_3_accuracy: 0.5474 - val_dens
e_3_loss: 1.5112 - val_dense_4_accuracy: 0.5495 - val_dense_4_loss: 1.4910 - va
l_loss: 2.4338
Epoch 49/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 41s 126ms/step - dense_1_accuracy: 0.5160 - dens
e_1_loss: 1.5381 - dense_3_accuracy: 0.6008 - dense_3_loss: 1.2529 - dense_4_ac
curacy: 0.6300 - dense_4_loss: 1.1508 - loss: 1.9881 - val_dense_1_accuracy:
0.4990 - val_dense_1_loss: 1.6291 - val_dense_3_accuracy: 0.5427 - val_dens
e_3_loss: 1.5346 - val_dense_4_accuracy: 0.5495 - val_dense_4_loss: 1.5315 - va
l_loss: 2.4816
Epoch 50/50
184/184 ━━━━━━━━━━━━━━━━━━━━ 25s 138ms/step - dense_1_accuracy: 0.5221 - dens
e_1_loss: 1.5107 - dense_3_accuracy: 0.6082 - dense_3_loss: 1.2424 - dense_4_ac
curacy: 0.6536 - dense_4_loss: 1.0952 - loss: 1.9212 - val_dense_1_accuracy:
0.5078 - val_dense_1_loss: 1.6205 - val_dense_3_accuracy: 0.5433 - val_dens
e_3_loss: 1.5023 - val_dense_4_accuracy: 0.5549 - val_dense_4_loss: 1.4740 - va
l_loss: 2.4066
```

In [ ]:

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Function to get true labels and predicted labels from the validation dataset
def get_predictions_and_labels(dataset, model):
    true_labels = []
    pred_labels = []

    print("Making predictions on validation dataset...")

    for batch_num, (images, labels) in enumerate(dataset):
        batch_true_labels = labels[0].numpy()
        true_labels.extend(batch_true_labels)

        predictions = model.predict(images, verbose=0)

        # Use main classifier output (first output)
        main_predictions = predictions[0]
        batch_pred_labels = np.argmax(main_predictions, axis=1)
        pred_labels.extend(batch_pred_labels)

        # Print progress
        if batch_num % 10 == 0:
            print(f"Processed batch {batch_num + 1}")
```

```
    return np.array(true_labels), np.array(pred_labels)

# Get predictions on validation dataset
y_true, y_pred = get_predictions_and_labels(validation_dataset, googlenet)

# Calculate accuracy
accuracy = np.mean(y_true == y_pred)
print(f"\nValidation Accuracy: {accuracy:.4f}")

# Create confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Get class names (you may want to replace these with actual sport names)
class_names = [f'Class_{i}' for i in range(num_classes)]

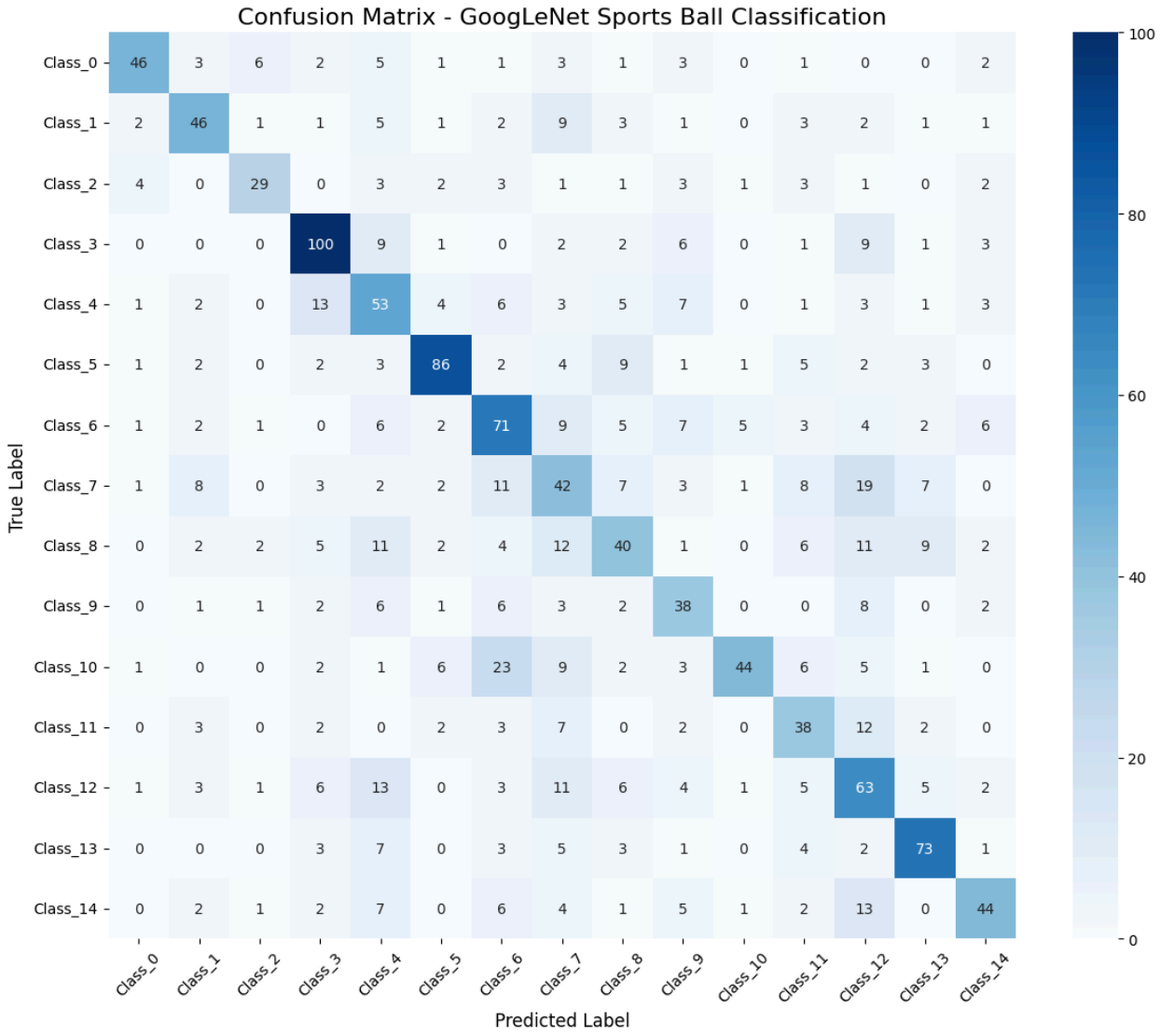
# Plot confusion matrix using matplotlib and seaborn
plt.figure(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - GoogLeNet Sports Ball Classification', fontsize=
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# Print classification report
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=class_names))

# Calculate per-class accuracy
print("\nPer-class Accuracy:")
for i in range(num_classes):
    class_mask = y_true == i
    if np.sum(class_mask) > 0:
        class_accuracy = np.mean(y_pred[class_mask] == y_true[class_mask])
        print(f"{class_names[i]}: {class_accuracy:.4f}")
```

```
Making predictions on validation dataset...
Processed batch 1
Processed batch 11
Processed batch 21
Processed batch 31
Processed batch 41

Validation Accuracy: 0.5549
```


Confusion Matrix - GoogLeNet Sports Ball Classification
True Label
Predicted Label
Class_0 46 3 6 2 5 1 1 3 1 3 0 1 0 0 2
Class_1 2 46 1 1 5 1 2 9 3 1 0 3 2 1 1
Class_2 4 0 29 0 3 2 3 1 1 3 1 3 1 0 2
Class_3 0 0 0 100 9 1 0 2 2 6 0 1 9 1 3
Class_4 1 2 0 13 53 4 6 3 5 7 0 1 3 1 3
Class_5 1 2 0 2 3 86 2 4 9 1 1 5 2 3 0
Class_6 1 2 1 0 6 2 71 9 5 7 5 3 4 2 6
Class_7 1 8 0 3 2 2 11 42 7 3 1 8 19 7 0
Class_8 0 2 2 5 11 2 4 12 40 1 0 6 11 9 2
Class_9 0 1 1 2 6 1 6 3 2 38 0 0 8 0 2
Class_10 1 0 0 2 1 6 23 9 2 3 44 6 5 1 0
Class_11 0 3 0 2 0 2 3 7 0 2 0 38 12 2 0
Class_12 1 3 1 6 13 0 3 11 6 4 1 5 63 5 2
Class_13 0 0 0 3 7 0 3 5 3 1 0 4 2 73 1
Class_14 0 2 1 2 7 0 6 4 1 5 1 2 13 0 44
Class_0 Class_1 Class_2 Class_3 Class_4 Class_5 Class_6 Class_7 Class_8 Class_9 Class_10 Class_11 Class_12 Class_13 Class_14
100
80
60
40
20
0

```
Classification Report:
              precision    recall  f1-score   support

     Class_0       0.79      0.62      0.70        74
     Class_1       0.62      0.59      0.61        78
     Class_2       0.69      0.55      0.61        53
     Class_3       0.70      0.75      0.72       134
     Class_4       0.40      0.52      0.45       102
     Class_5       0.78      0.71      0.74       121
     Class_6       0.49      0.57      0.53       124
     Class_7       0.34      0.37      0.35       114
     Class_8       0.46      0.37      0.41       107
     Class_9       0.45      0.54      0.49        70
    Class_10       0.81      0.43      0.56       103
    Class_11       0.44      0.54      0.48        71
    Class_12       0.41      0.51      0.45       124
    Class_13       0.70      0.72      0.71       102
    Class_14       0.65      0.50      0.56        88

    accuracy                           0.55      1465
   macro avg       0.58      0.55      0.56      1465
weighted avg       0.58      0.55      0.56      1465


Per-class Accuracy:
Class_0: 0.6216
Class_1: 0.5897
Class_2: 0.5472
Class_3: 0.7463
Class_4: 0.5196
Class_5: 0.7107
Class_6: 0.5726
Class_7: 0.3684
Class_8: 0.3738
Class_9: 0.5429
Class_10: 0.4272
Class_11: 0.5352
Class_12: 0.5081
Class_13: 0.7157
Class_14: 0.5000
```

In [ ]:

```python
# Additional evaluation metrics
from sklearn.metrics import precision_recall_fscore_support

# Calculate precision, recall, and F1-score
precision, recall, f1, support = precision_recall_fscore_support(
    y_true, y_pred, average=None, labels=range(num_classes)
)

# Create a summary dataframe
import pandas as pd

results_df = pd.DataFrame({
    'Class': class_names,
```

```
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'Support': support
})

print("Detailed Performance Metrics:")
print(results_df.round(4))

# Calculate macro and weighted averages
macro_avg = precision_recall_fscore_support(y_true, y_pred, average='macro')
weighted_avg = precision_recall_fscore_support(y_true, y_pred, average='weight

print(f"\nMacro Average - Precision: {macro_avg[0]:.4f}, Recall: {macro_avg[1]
print(f"Weighted Average - Precision: {weighted_avg[0]:.4f}, Recall: {weighted
```

```
Detailed Performance Metrics:
       Class  Precision  Recall  F1-Score  Support
0    Class_0     0.7931  0.6216    0.6970       74
1    Class_1     0.6216  0.5897    0.6053       78
2    Class_2     0.6905  0.5472    0.6105       53
3    Class_3     0.6993  0.7463    0.7220      134
4    Class_4     0.4046  0.5196    0.4549      102
5    Class_5     0.7818  0.7107    0.7446      121
6    Class_6     0.4931  0.5726    0.5299      124
7    Class_7     0.3387  0.3684    0.3529      114
8    Class_8     0.4598  0.3738    0.4124      107
9    Class_9     0.4471  0.5429    0.4903       70
10  Class_10     0.8148  0.4272    0.5605      103
11  Class_11     0.4419  0.5352    0.4841       71
12  Class_12     0.4091  0.5081    0.4532      124
13  Class_13     0.6952  0.7157    0.7053      102
14  Class_14     0.6471  0.5000    0.5641       88

Macro Average - Precision: 0.5825, Recall: 0.5519, F1: 0.5591
Weighted Average - Precision: 0.5785, Recall: 0.5549, F1: 0.5588
```

In [ ]:
```
def plot_loss_comparison_manual(history):
    """
    Plot training loss vs validation loss with manually specified keys
    """
    # Create figure with subplots
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    fig.suptitle('Training vs Validation Loss - GoogLeNet Sports Ball Classifi

    # Get all available loss keys (excluding 'loss' and validation keys)
    loss_keys = [k for k in history.history.keys() if 'loss' in k and 'val' no
    print(f"Available loss keys: {loss_keys}")

    # Try to plot with available keys
    if len(loss_keys) >= 3:
        # Main classifier loss (first output layer)
        main_key = loss_keys[0]
        axes[0, 0].plot(history.history[main_key], 'b-', label='Training Loss'
```

```
        axes[0, 0].plot(history.history[f'val_{main_key}'], 'r-', label='Valid
        axes[0, 0].set_title(f'Main Classifier Loss ({main_key})', fontsize=14
        axes[0, 0].set_xlabel('Epoch')
        axes[0, 0].set_ylabel('Loss')
        axes[0, 0].legend()
        axes[0, 0].grid(True, alpha=0.3)

        # First auxiliary classifier loss
        aux1_key = loss_keys[1]
        axes[0, 1].plot(history.history[aux1_key], 'b-', label='Training Loss'
        axes[0, 1].plot(history.history[f'val_{aux1_key}'], 'r-', label='Valid
        axes[0, 1].set_title(f'Auxiliary Classifier 1 Loss ({aux1_key})', font
        axes[0, 1].set_xlabel('Epoch')
        axes[0, 1].set_ylabel('Loss')
        axes[0, 1].legend()
        axes[0, 1].grid(True, alpha=0.3)

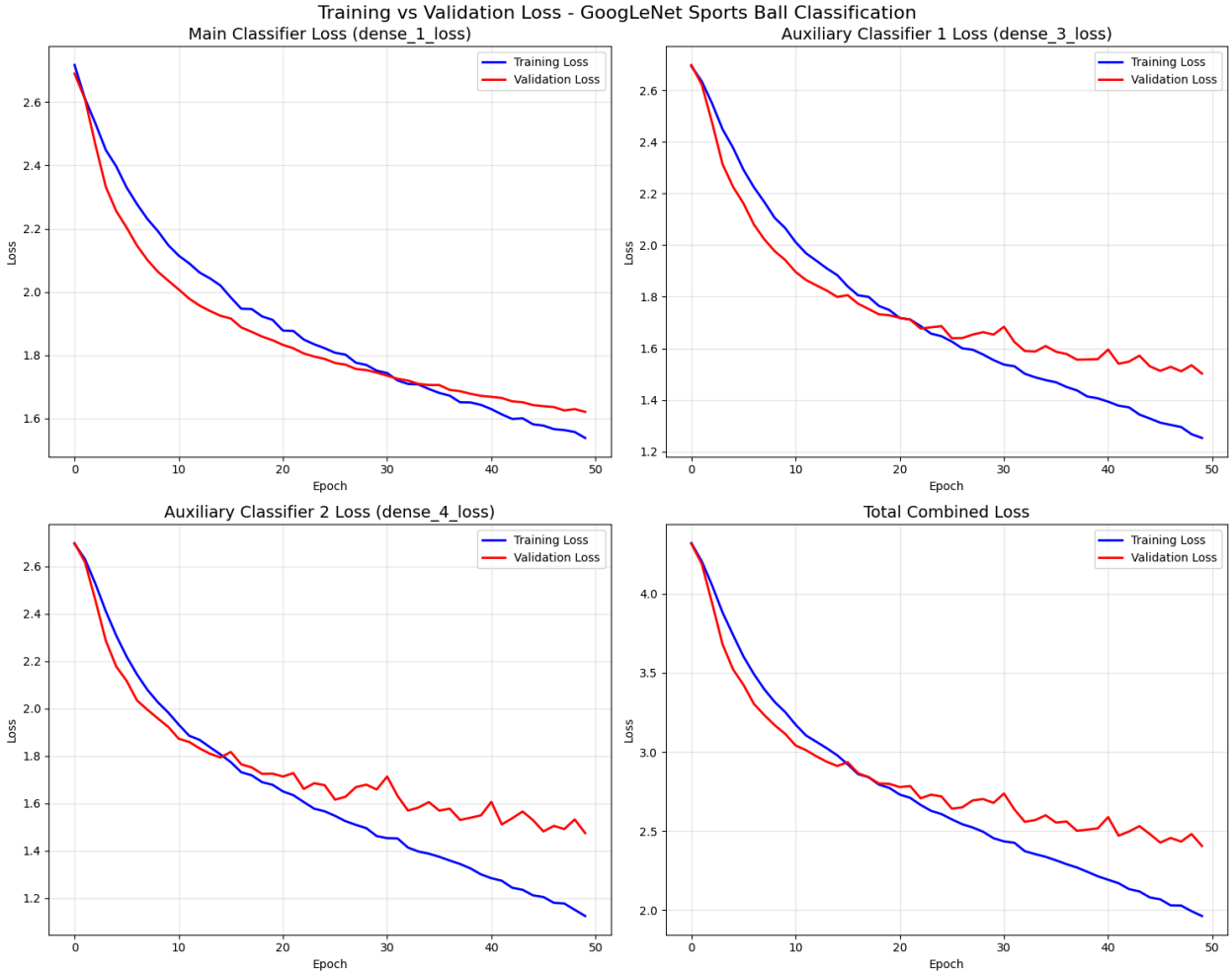
        # Second auxiliary classifier loss
        aux2_key = loss_keys[2]
        axes[1, 0].plot(history.history[aux2_key], 'b-', label='Training Loss'
        axes[1, 0].plot(history.history[f'val_{aux2_key}'], 'r-', label='Valid
        axes[1, 0].set_title(f'Auxiliary Classifier 2 Loss ({aux2_key})', font
        axes[1, 0].set_xlabel('Epoch')
        axes[1, 0].set_ylabel('Loss')
        axes[1, 0].legend()
        axes[1, 0].grid(True, alpha=0.3)

    # Combined/Total loss (this should always be available)
    axes[1, 1].plot(history.history['loss'], 'b-', label='Training Loss', line
    axes[1, 1].plot(history.history['val_loss'], 'r-', label='Validation Loss'
    axes[1, 1].set_title('Total Combined Loss', fontsize=14)
    axes[1, 1].set_xlabel('Epoch')
    axes[1, 1].set_ylabel('Loss')
    axes[1, 1].legend()
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Call the function
plot_loss_comparison_manual(history)
```

```
Available loss keys: ['dense_1_loss', 'dense_3_loss', 'dense_4_loss']
```



In [ ]:
```
def plot_simple_loss(history):
    plt.figure(figsize=(10, 6))

    plt.plot(history.history['loss'], 'b-', label='Training Loss', linewidth=2
    plt.plot(history.history['val_loss'], 'r-', label='Validation Loss', linew

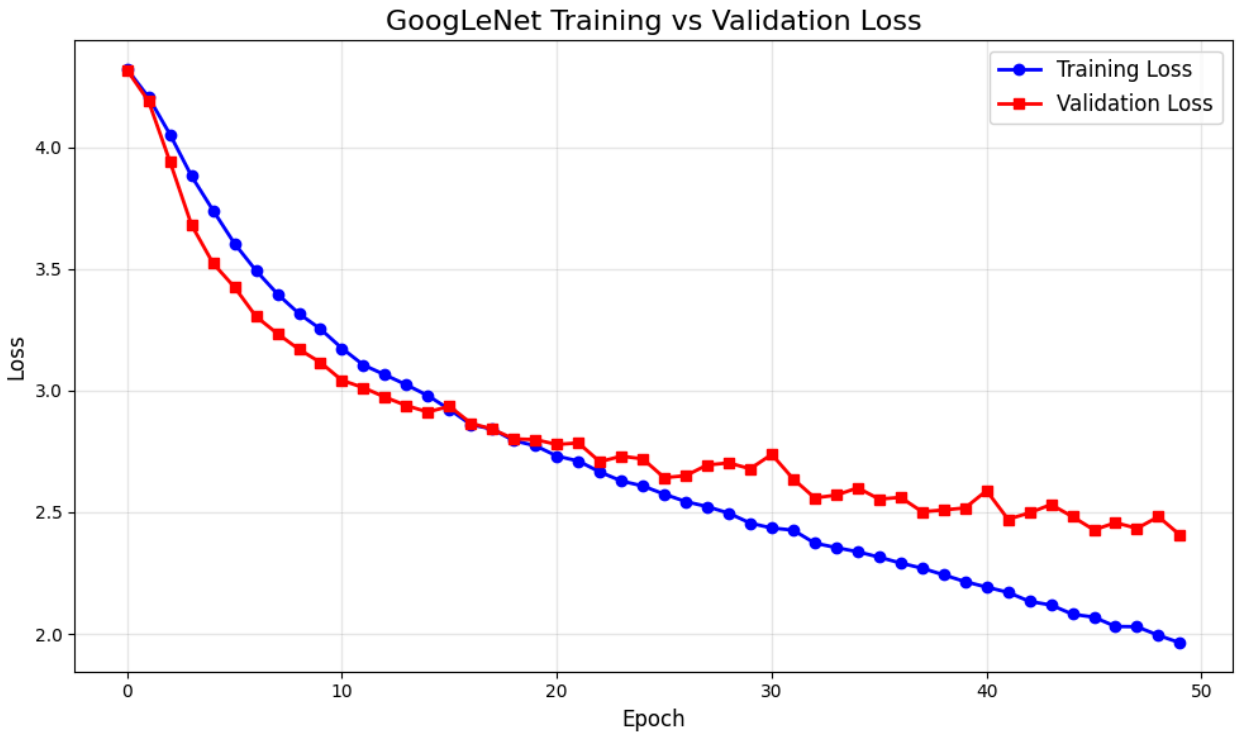
    plt.title('GoogLeNet Training vs Validation Loss', fontsize=16)
    plt.xlabel('Epoch', fontsize=12)
    plt.ylabel('Loss', fontsize=12)
    plt.legend(fontsize=12)
    plt.grid(True, alpha=0.3)

    # Print final values
    print(f"Final Training Loss: {history.history['loss'][-1]:.4f}")
    print(f"Final Validation Loss: {history.history['val_loss'][-1]:.4f}")

    plt.tight_layout()
    plt.show()

# This should definitely work
plot_simple_loss(history)
```

```
Final Training Loss: 1.9636
Final Validation Loss: 2.4066
```


GoogLeNet Training vs Validation Loss
Training Loss
Validation Loss
Loss
Epoch
4.0
3.5
3.0
2.5
2.0
0
10
20
30
40
50

In [ ]: